

Natural Language Processing
TP 3 – Support Vector Machine
Appliquer les SVM pour prédire la catégorie des actualités BBC
TP noté (Évaluation continue)

Vous devez soumettre vos livrables sur MOODLE, en respectant la date limite fixée par votre formateur. Soumettez votre propre travail. La triche ne sera pas tolérée et sera pénalisée. Cette activité peut être réalisée en groupe (jusqu'à 4 membres par groupe).

Support Vector Machines (SVMs)

Support Vector Machines (SVMs) ou les Machines à Vecteurs de Support sont une classe de modèles d'apprentissage supervisé largement utilisés pour les tâches de classification et de régression.

L'idée clé des SVM est de trouver l'hyperplan optimal qui sépare le mieux les points de données appartenant à différentes classes dans un espace à haute dimension.

Pour la classification de texte, les SVM sont particulièrement efficaces car les données textuelles sont souvent représentées dans un espace de caractéristiques à haute dimension (par exemple, via le bag-of-words ou les vecteurs TF-IDF), et les SVM peuvent gérer efficacement cette sparseur tout en maximisant la marge entre les classes.

Les SVM peuvent utiliser différents noyaux (ou kernels) comme les kernels linéaire, polynomial, fonction de base radiale, etc., pour capturer à la fois des relations linéaires et non-linéaires entre les caractéristiques. Dans le domaine du traitement automatique du langage naturel (NLP), les SVM sont fréquemment appliqués à des tâches telles que l'analyse de sentiment, la détection de spam, la classification thématique, et bien d'autres encore.

Objectifs d'apprentissage :

- À la fin de cette activité, vous serez capable de :
- Comprendre le concept et le principe de fonctionnement des Machines à Vecteurs de Support (SVM).
- Appliquer des techniques de prétraitement du texte pour des tâches de NLP en utilisant NLTK.
- Préparer les données textuelles pour l'apprentissage supervisé en combinant le titre et le contenu, en tokenisant, en lemmatisant et en supprimant les stop words.
- Convertir les données textuelles en représentations numériques à l'aide de la vectorisation TF-IDF.
- Entraîner et évaluer un modèle SVM pour des tâches de classification de texte.
- Interpréter les sorties du modèle et évaluer les performances de classification en utilisant des métriques appropriées.

Cas d'usage 3.1 : Classification des critiques de vin avec SVM

Nous reviendrons sur le cas d'usage des critiques de vin, où un modèle SVM a été appliqué. Reportez-vous à cet exemple pour vérifier comment le SVM a été implémenté pour classer les critiques de vin.

Le code source de cette tâche a été fourni dans le TP 2.2.

Cas d'usage 3.2 : Classification des catégories de nouvelles BBC avec SVM (Partie Notée du TP)

Le jeu de données BBC News inclut :

- **category**
C'est la variable cible (l'étiquette).
Elle indique le type d'article de presse (par exemple, business, politique, sport, technologie, divertissement).
- **filename**
Le nom de fichier original du document texte (par exemple, 163.txt).
Chaque fichier contient un article de presse. Il s'agit uniquement d'un identifiant, non utile pour la classification en elle-même.
- **title**
Le titre de l'article de presse.
Exemple : « US data sparks inflation worries ».
- **content**
Le texte principal de l'article.
Exemple : « Wholesale prices in the US rose at the fastest ... ».

Nous travaillerons avec les nouvelles BBC pour 2004-2005 disponibles sur :

<https://www.kaggle.com/hgultekin/bbcnewsarchive>

Objectif principal de ce TP :

Cette tâche consiste en un problème de classification supervisée de texte. L'entrée est le texte de l'article de presse (une combinaison du titre et du contenu), et la sortie est l'étiquette de catégorie correspondante.

Le titre et le contenu sont fusionnés dans un seul champ appelé `raw_text` pour représenter l'article complet.

L'objectif est d'entraîner un modèle de machine learning capable de prédire la catégorie correcte à partir de ce texte.

Consignes :

Prétraitement du test en utilisant NLTK:

Cette procédure consiste en les points suivants :

- Combiner le titre et le contenu de l'article dans un seul champ.
- Mettre le texte en minuscules, le découper en tokens (mots).
- Ne conserver que les mots composés de lettres (en supprimant ainsi la ponctuation et les nombres).
- Appliquer la lemmatisation.
- Supprimer les stop words.

Vous devez utiliser la bibliothèque NLTK pour le prétraitement du texte :

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
```

Vous pouvez utiliser les commandes suivantes :

```
nltk.download('punkt') #this tokenizer divides a text into a list of sentences
nltk.download('wordnet') #it is a large word database of English Nouns, Adjectives,
Adverbs and Verbs
nltk.download('stopwords') #commonly used word
nltk.download('omw-1.4') #provides access to open wordnets in a variety of
languages, all linked to a collaborative Interlingual Index
```

Modélisation SVM :

Cette procédure consiste en les étapes suivantes :

- Séparer les données en ensembles d'entraînement et de test.
- Vectoriser les textes avec
TfidfVectorizer() from sklearn.feature_extraction.text
- Entraîner un classificateur Support Vector Machine sur les données d'entraînement.
- Évaluer le modèle sur l'ensemble de test.

NB : n'oubliez pas d'ajouter des commentaires internes d'interprétation des résultats et d'explication du code à chaque étape. Ceci est obligatoire et sera noté.

Livrables attendus :

- Le jeu de données
- Un Notebook Python comprenant :
 - La description de votre cas d'usage
 - Le code source
 - Les sorties (tableaux, graphiques, résultats imprimés)
 - Des commentaires internes expliquant et interprétant les résultats

Veuillez respecter la date limite de soumission des livrables sur **MOODLE**.

Critères d'évaluation :

1. Chargement des données (5 points)

- Importer le fichier CSV ou TXT contenant les articles (titre + contenu + catégorie).

2. Prétraitement du texte (Text Preprocessing) – Workflow détaillé

- **Combiner le titre et le contenu (5 points)**
 - Créer un champ unique `raw_text = titre + contenu`
- **Conversion en minuscules (5 points)**
 - `raw_text.lower()`
 - Permet de normaliser le texte et éviter que « Wine » et « wine » soient traités différemment
- **Tokenization (découpage en mots) (5 points)**
 - `tokens = word_tokenize(raw_text)`
 - Transforme le texte en une **liste de mots**
- **Filtrage : ne garder que les lettres (5 points)**
 - Supprimer chiffres et ponctuation
 - Exemple : "wine123!" → "wine"
- **Suppression des stop words (5 points)**
 - Supprimer les mots très fréquents et peu informatifs (the, and, of...)
 - Utiliser `nlk.corpus.stopwords`
- **Lemmatisation (5 points)**
 - Réduire les mots à leur forme de base (running → run)
 - Utiliser `WordNetLemmatizer`
- **Résultat final (5 points)**
 - Une **liste de mots nettoyés et lemmatisés**, prête pour la vectorisation avec TF-IDF

3. Vectorisation (20 points)

- Convertir le texte en vecteurs numériques avec `TfidfVectorizer()`.

4. Séparation Train / Test (5 points)

- Diviser les données en ensemble d'entraînement et ensemble de test.

5. Entraînement du modèle SVM (15 points)

- Créer un classificateur **Support Vector Machine**.
- L'entraîner sur l'ensemble d'entraînement vectorisé.

6. Test et Évaluation du modèle (20 points)

- Prédire les catégories sur l'ensemble de test.
- Calculer les métriques de performance (accuracy, precision, recall...).
- Commenter les résultats et interpréter les performances.
- Identifier éventuellement les erreurs ou biais dans la classification.

GOOD LUCK!